
🌳 1. Intuition: What is a Binary Tree?

Imagine a **family tree** or **organization hierarchy**:

- One root person (CEO / ancestor)
- Each person has at most **2 children**
- Data flows **top → down**

A **Binary Tree** is exactly that:

A hierarchical data structure where each node has **at most two children**:

- **Left child**
- **Right child**

🔍 Visual Intuition

🧠 Key Terminology

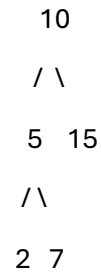
- **Root** → top node
- **Leaf** → node with no children
- **Height** → longest path from root to leaf
- **Depth** → distance from root
- **Subtree** → any node + its children

🎄 2. Binary Tree Structure (Conceptual Model)

Each node contains:

```
struct Node {
  int data;
  Node* left;
  Node* right;
};
```

Think of it as:



⚙️ 3. Operations on Binary Tree

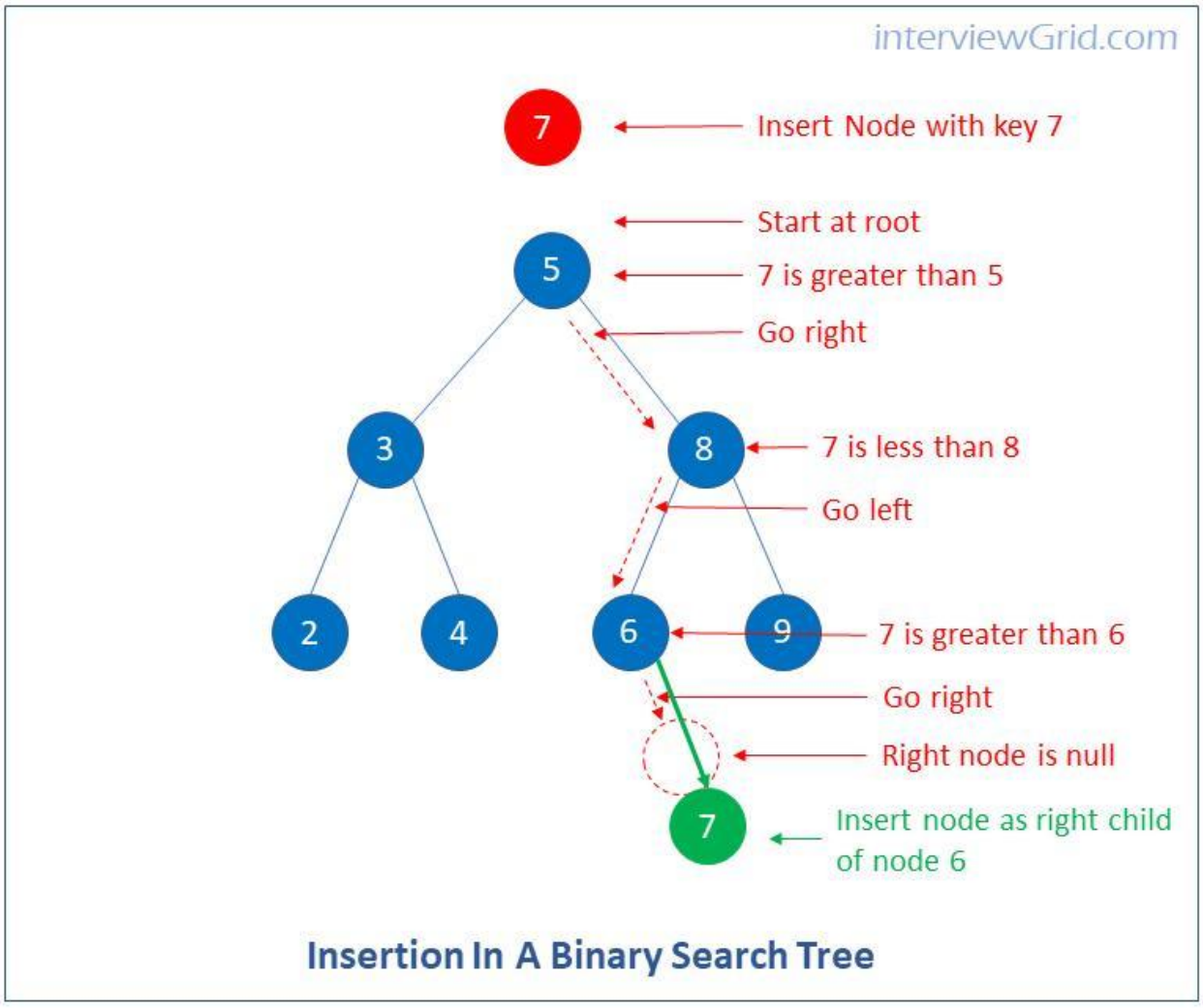
We'll go step by step:

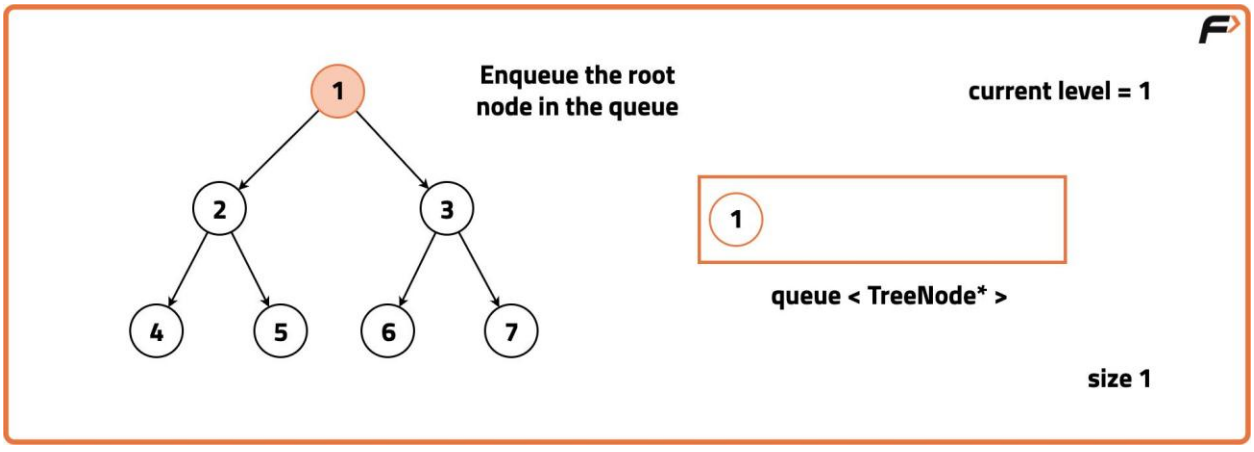
🧩 A. Insertion

💡 Idea:

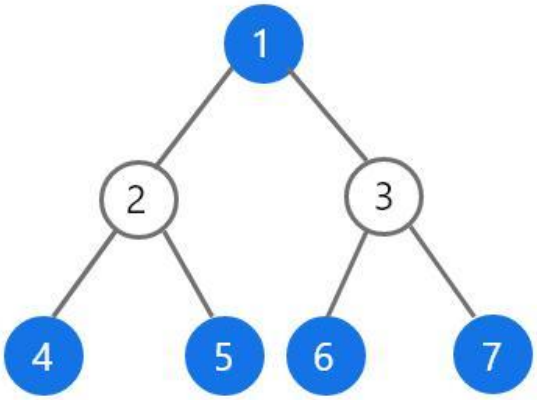
- Insert level-by-level (like filling seats in a classroom)

Visualization


interviewGrid.com
7
Insert Node with key 7
Start at root
5
7 is greater than 5
Go right
3
8
7 is less than 8
Go left
2
4
6
9
7 is greater than 6
Go right
Right node is null
7
Insert node as right child of node 6
Insertion In A Binary Search Tree

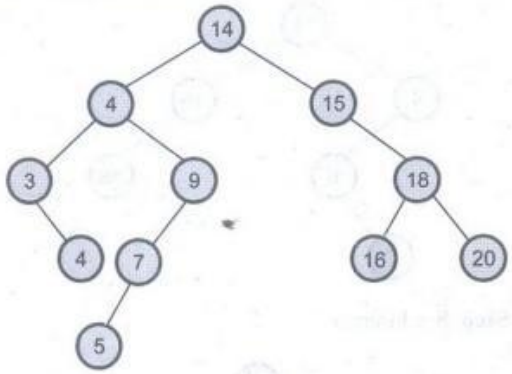

1
2
3
4
5
6
7
Enqueue the root node in the queue
current level = 1
1
queue < TreeNode* >
size 1

Level Order Traversal of Binary Tree

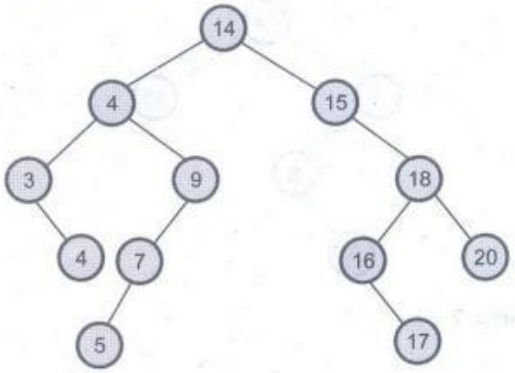


Level Order : 1, 2, 3, 4, 5, 6,7

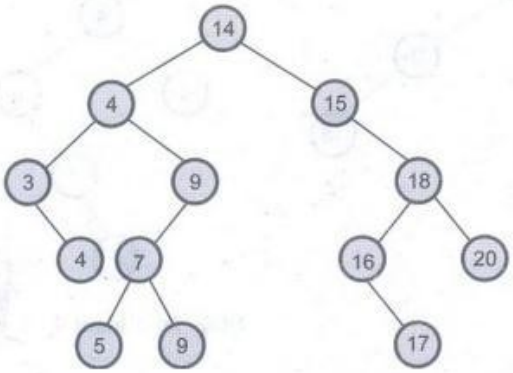
Step 11 : Insert 20



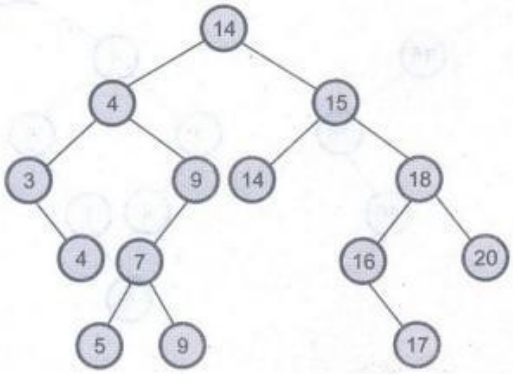
Step 12 : Insert 17



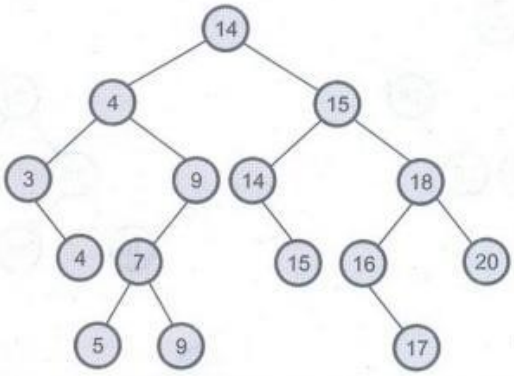
Step 13 : Insert 9

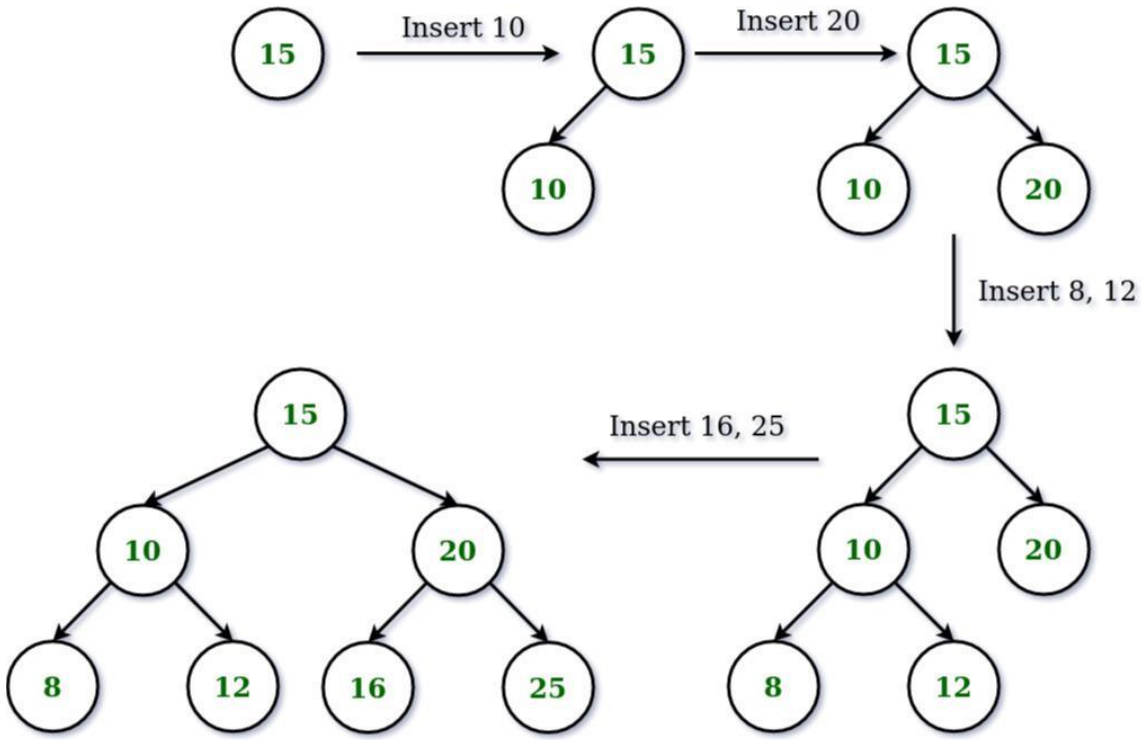


Step 14 : Insert 14

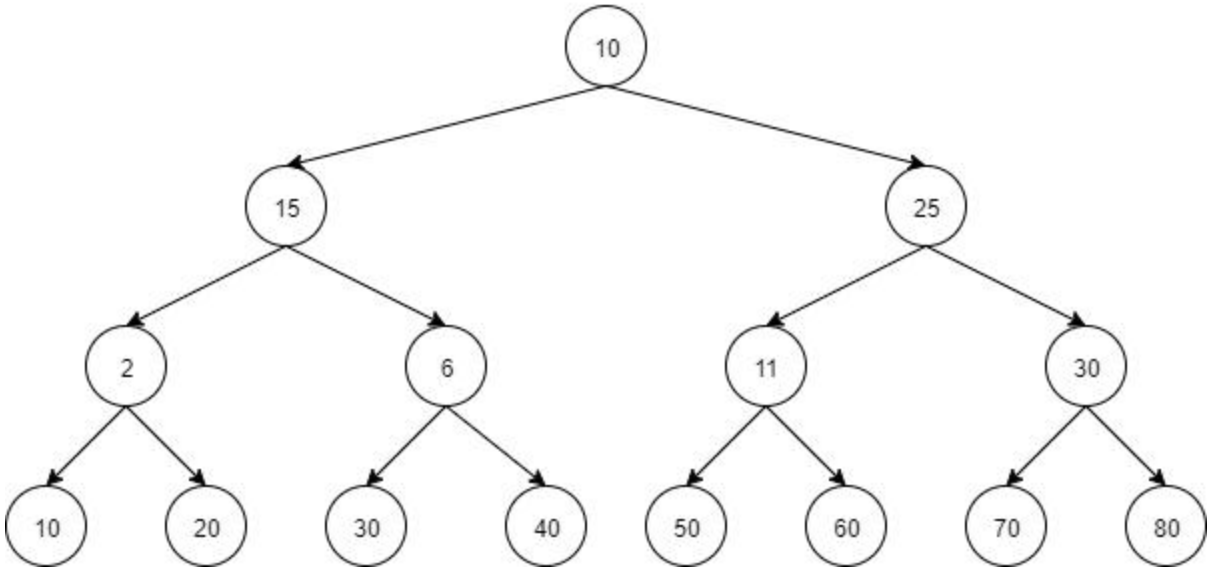


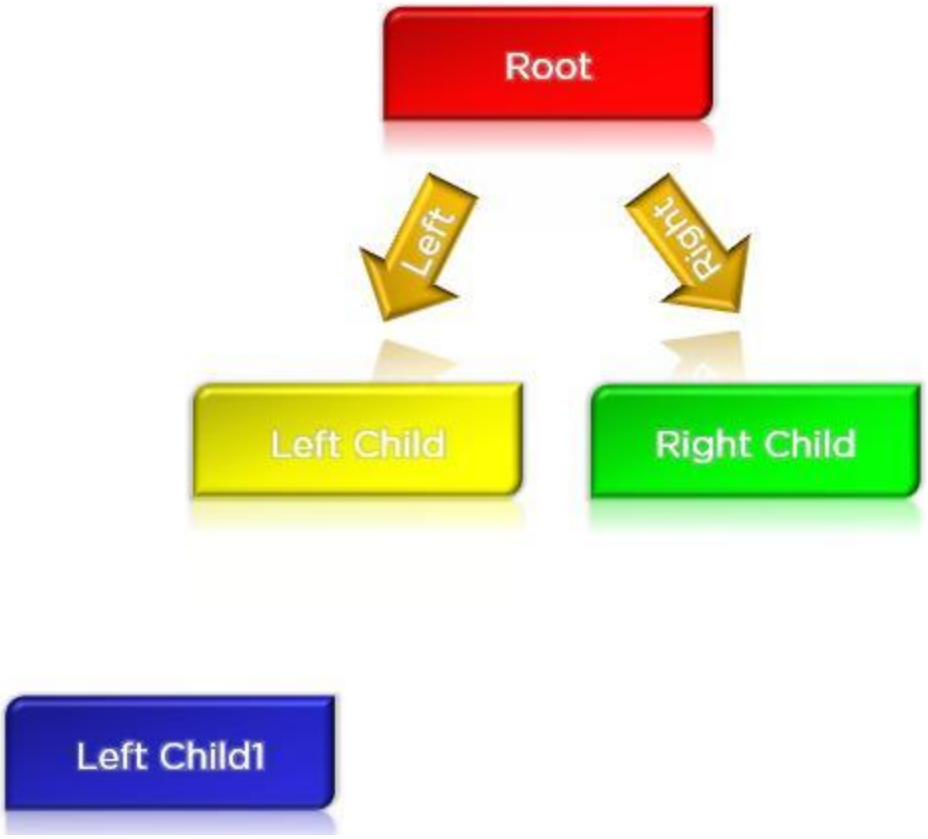
Step 15

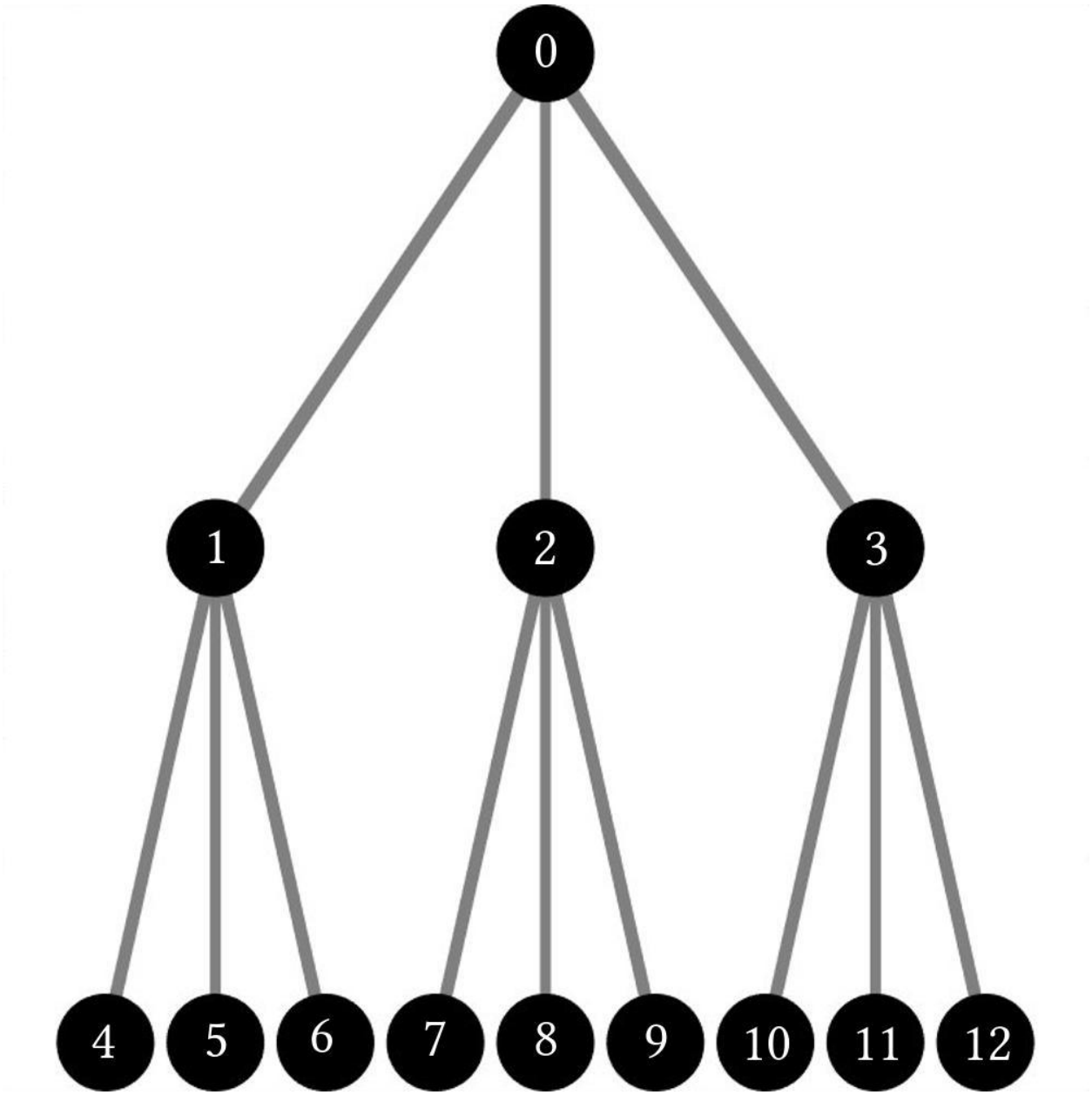




Complete Binary Tree




Root
Left
Right
Left Child
Right Child
Left Child1



🧠 **Logic:**

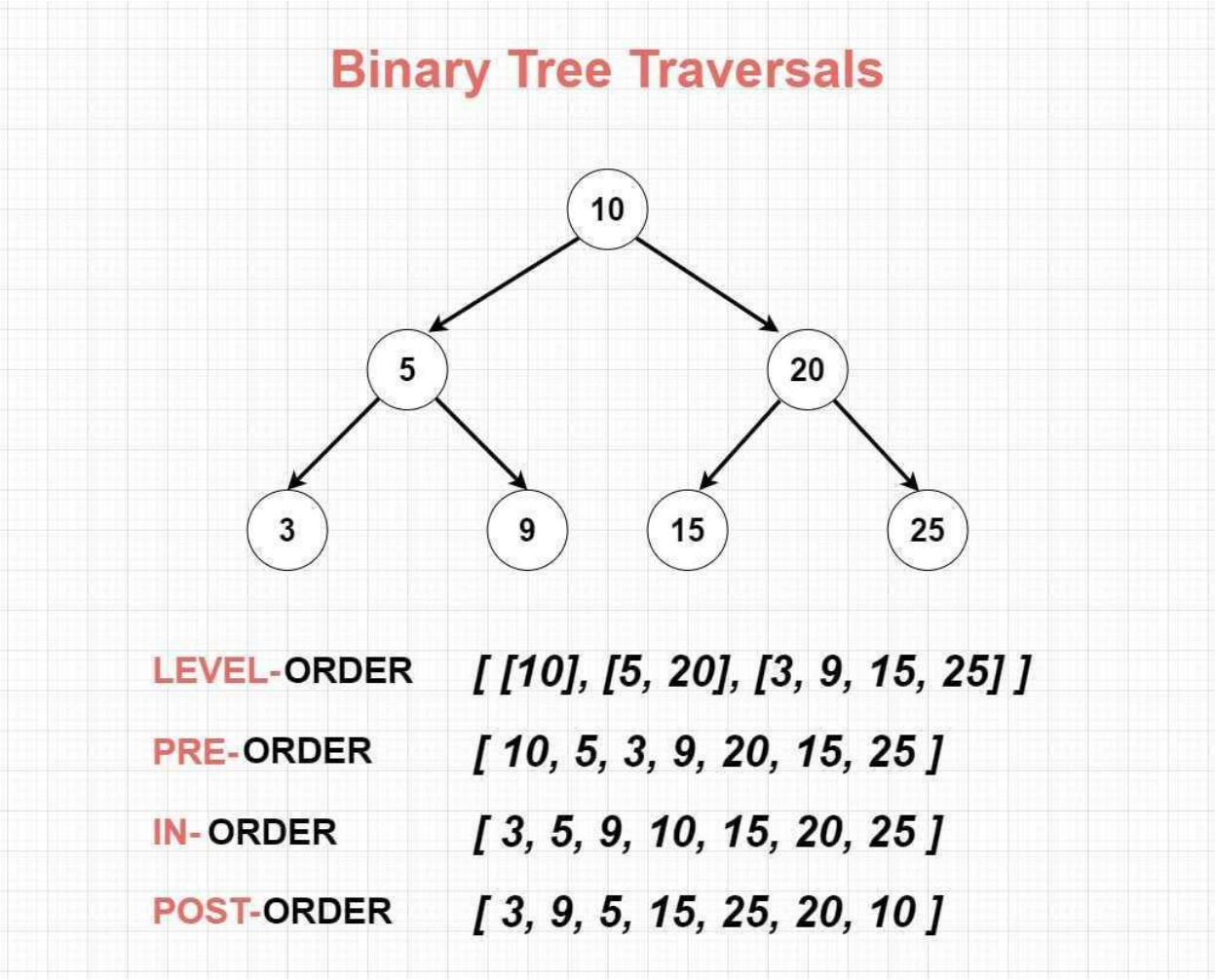
1. Use **queue (BFS)**
2. Find first empty left/right
3. Insert there

🔍 **B. Traversals (VERY IMPORTANT)**

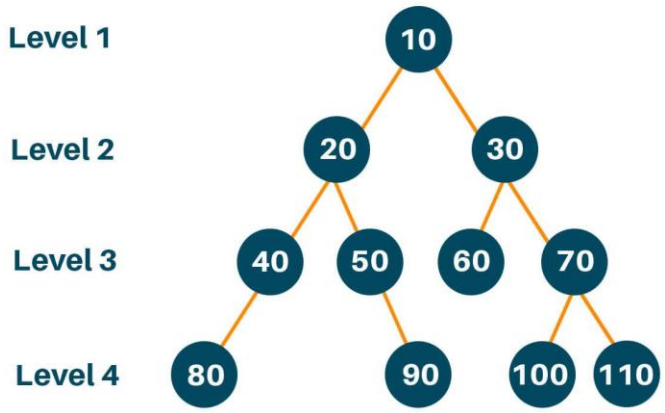
Types:

1. **Inorder (LNR)** → Left → Node → Right
2. **Preorder (NLR)** → Node → Left → Right
3. **Postorder (LRN)** → Left → Right → Node
4. **Level Order (BFS)**

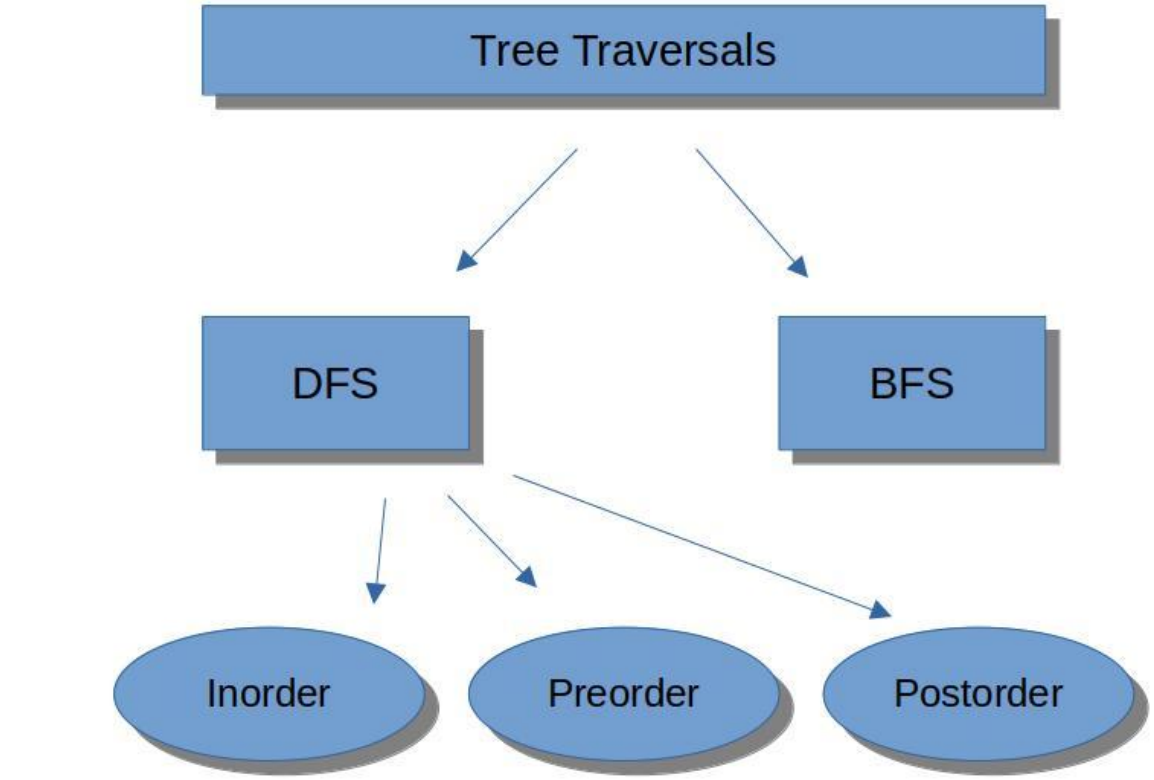
Visualization

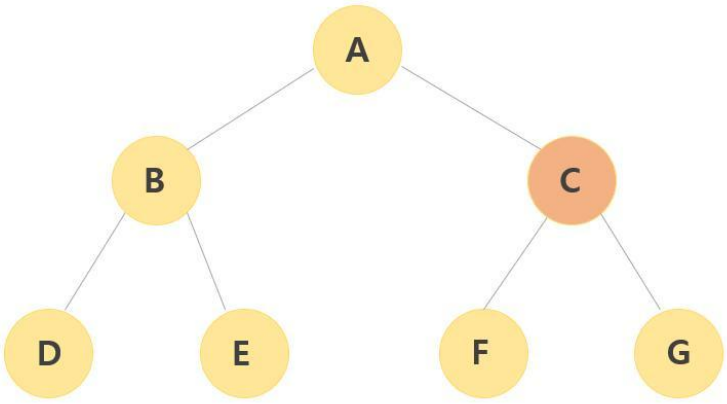


Level Order Traversal of a Binary Tree



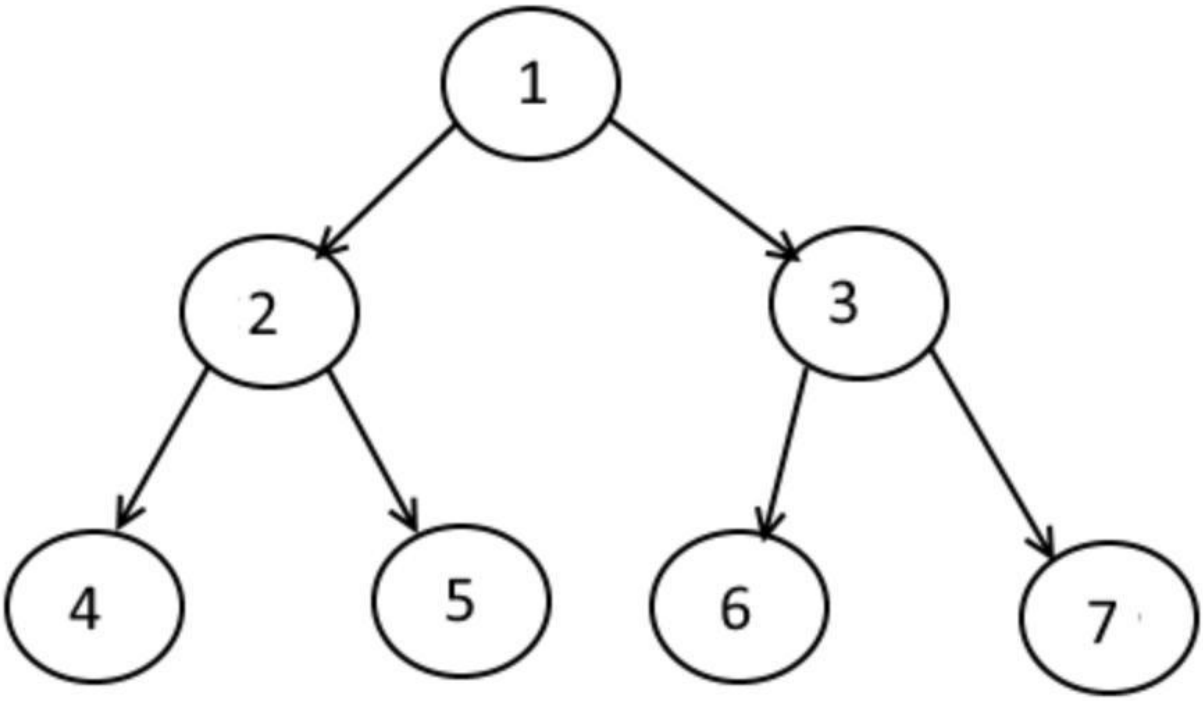
At level 1: [10]
At level 2: [20, 30]
At level 3: [40, 50, 60, 70]
At level 4: [80, 90, 100, 110]


Tree Traversals
DFS
BFS
Inorder
Preorder
Postorder

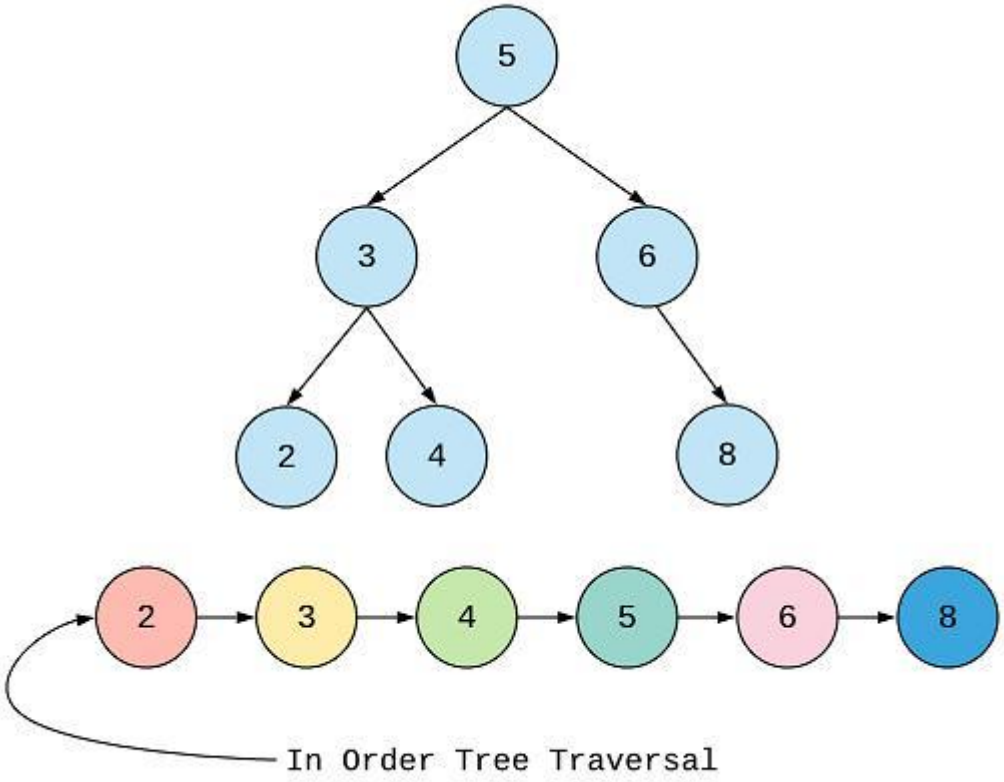

A
B
C
D
E
F
G


C
Stack


List
D
B
E
A



Inorder Traversal: 4 2 5 1 6 3 7
Preorder Traversal: 1 2 4 5 3 6 7
Postorder Traversal: 7 6 3 5 4 2 1
Breadth-First Search: 1 2 3 4 5 6 7
Depth-First Search: 1 2 4 5 3 6 7



❌ C. Deletion

💡 **Idea (General Binary Tree):**

1. Find **node to delete**
2. Replace with **deepest node**
3. Delete deepest node

🔎 D. Search

- Traverse tree (DFS or BFS)
- Compare values

📏 E. Height Calculation

- Recursively:

height = 1 + max(left, right)

🧱 4. High-Level Architecture (C++ Thinking)

Core Components:

Component	Purpose
Node struct	Tree building block
createNode()	Memory allocation
insert()	Add nodes
traversal()	Print/process
deleteNode()	Remove nodes
search()	Find value
height()	Measure tree

💻 5. C++ Implementation

🔹 Node Definition

```
#include <iostream>
#include <queue>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
```

```
    left = right = NULL;
  }
};
```

- **Insert (Level Order)**

```
Node* insert(Node* root, int val) {
  if (!root) return new Node(val);

  queue<Node*> q;
  q.push(root);

  while (!q.empty()) {
    Node* temp = q.front();
    q.pop();

    if (!temp->left) {
      temp->left = new Node(val);
      break;
    } else {
      q.push(temp->left);
    }

    if (!temp->right) {
      temp->right = new Node(val);
      break;
    } else {
      q.push(temp->right);
    }
  }
```

```
    return root;
}
```

- **Traversals**

Inorder

```
void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

Preorder

```
void preorder(Node* root) {
    if (!root) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

Postorder

```
void postorder(Node* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

Level Order

```
void levelOrder(Node* root) {
    if (!root) return;
```

```
    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* temp = q.front();
        q.pop();

        cout << temp->data << " ";

        if (temp->left) q.push(temp->left);
        if (temp->right) q.push(temp->right);
    }
}
```

- **Search**

```
bool search(Node* root, int key) {
    if (!root) return false;
    if (root->data == key) return true;

    return search(root->left, key) || search(root->right, key);
}
```

- **Height**

```
int height(Node* root) {
    if (!root) return 0;
    return 1 + max(height(root->left), height(root->right));
}
```

- **Delete Node**

```
Node* deleteDeepest(Node* root, Node* d_node) {
  queue<Node*> q;
  q.push(root);

  while (!q.empty()) {
    Node* temp = q.front();
    q.pop();

    if (temp->left) {
      if (temp->left == d_node) {
        temp->left = NULL;
        delete d_node;
        return root;
      } else q.push(temp->left);
    }

    if (temp->right) {
      if (temp->right == d_node) {
        temp->right = NULL;
        delete d_node;
        return root;
      } else q.push(temp->right);
    }
  }
  return root;
}

Node* deleteNode(Node* root, int key) {
  if (!root) return NULL;
```

```
if (!root->left && !root->right) {
   if (root->data == key) return NULL;
   else return root;
}

queue<Node*> q;
q.push(root);

Node* key_node = NULL;
Node* temp;

while (!q.empty()) {
   temp = q.front();
   q.pop();

   if (temp->data == key)
      key_node = temp;

   if (temp->left) q.push(temp->left);
   if (temp->right) q.push(temp->right);
}

if (key_node) {
   int x = temp->data;
   deleteDeepest(root, temp);
   key_node->data = x;
}
```

```
    return root;
}
```

🚀 6. Real Backend Use Cases

Binary Trees are not just academic—they power real systems:

🧠 A. Databases (Indexing)

- **Binary Search Trees (BST) / B-Trees**
- Used in:
    - MySQL indexes
    - PostgreSQL indexes

👉 Fast search: **O(log n)**

⚡ B. Caching Systems

- Trees used in:
    - LRU cache variants
    - Memory hierarchies

🌐 C. Web Routing / Parsing

- Expression trees
- JSON/XML parsing trees

🤖 D. AI / ML

- **Decision Trees**
- Random Forests

🔐 E. Compilers

- Abstract Syntax Tree (AST)

- Expression evaluation

📦 F. File Systems

- Directory structure = tree

🎯 Final Mental Model

Think of Binary Tree as:

"A recursive structure where every node is itself a smaller tree"

🔥 If You Want Next Step

I can take you deeper into:

- Binary Search Tree (BST) vs Binary Tree
- Balanced Trees (AVL, Red-Black)
- Interview problems (VERY important 🔥)
- STL alternatives (map, set internals)

Just tell me 👍